Object Oriented Programming:

Java Programming



Informatics Engineering, UNRAM

August, 2026.

Why Java? Why OOP? Why Now?

🏢 Industry

99% of enterprise organizations use Java. 23+ million active developers worldwide. Powers finance, healthcare, e-commerce, cloud systems, and Android.

🎓 Academia

Same OOP learning path used at other campus (UC Berkeley, CMU, and Cornell): classes → inheritance → polymorphism → GUI → projects.

🤖 AI Era

Modern AI systems are built on OOP foundations. Java is used to integrate AI APIs (OpenAI, Anthropic). Understanding classes and methods is how you build AI-powered apps.

💡 AI tools exist and you may already use them. Course rule: you must be able to explain every line of code you submit.



Object Oriented Programming : Core Concept

Object Oriented?



Attribute (State):

Tires, Steering Wheel, Brake Pedal, Gas Pedal , Color, Year of Production

Behavior:

How to Start the Engine
How to Start a Car
How to Reverse a Car

Attribute → Variable(Member)
Behavior → Method (Function)
Behavior Using Attributes

What is Object Oriented?

Think of a Car 🚗

📋 Attributes (State / Data)

Color · Year of Production · Fuel Level · Current Speed · Number of Doors

⚙️ Behavior (Methods / Actions)

startEngine() · accelerate() · brake() · reverse()

In Code...

Attribute	→	Variable (Member field)
Behavior	→	Method (Function)
Key insight:		Methods often USE attributes e.g. accelerate() changes speed



SOFTWARE : OBJECT ORIENTED

- Software that built from classes and objects that interact with each other.
  - Class

    A static description of something.
  - Object

    Something created (instantiation) of a class.



Software: Object Oriented

Object-oriented software is built from classes and objects that interact with each other.

Class

A blueprint or template : it describes the structure and behavior of something, but is not a real thing by itself.

Object

An instance of a class : a real, concrete thing created from that blueprint, with its own actual data values.

Difference between Class and Object

- Class : concept and description from something
  - Class declares methods that can be used (called) by objects
- Object : an instance of a class , a real form (example) of a class
  - Objects have independent properties and can be used to call methods
- Examples of Classes and Objects :
  - Class: car
  - Object: Mr Dani's car, my car, red car



Difference Between Class and Object

CLASS (Blueprint)

- A concept and description of something
- Declares methods that objects can call
- Has no real data of its own, just structure
- **Example:**
  - Car (the concept of a car)

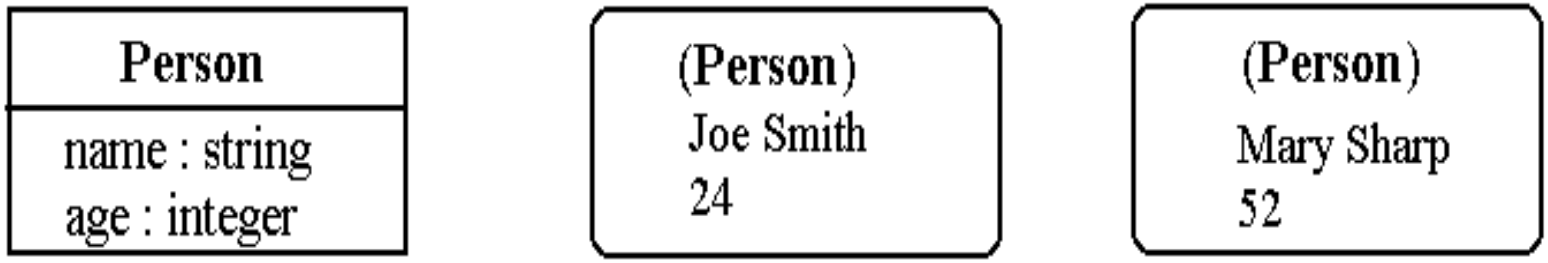
instantiation →

OBJECT (Instance)

- An instance of a class, a real thing
- Has its own independent data values
- Can call the methods defined in its class
- **Examples:**
  - myHondaCivic, redSportsCar, mrJokosCar

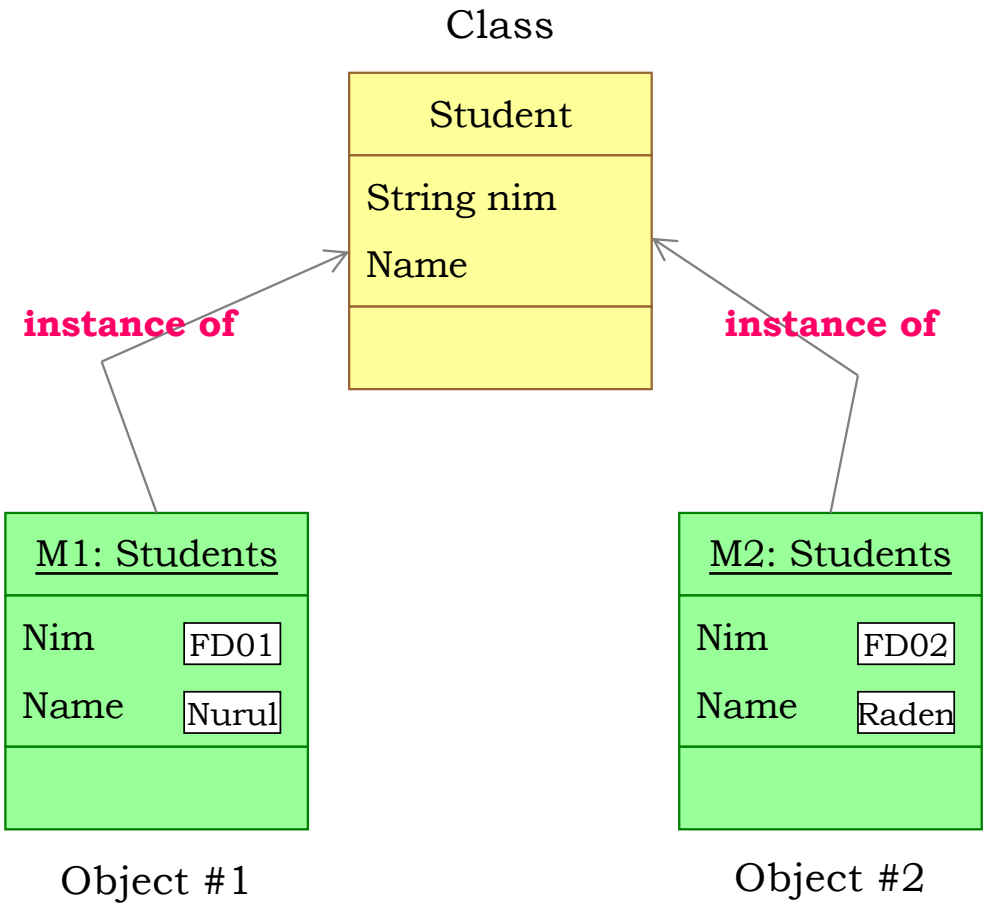
Difference between Class and Object

- Classes are like cake molds , where the cake produced from the cake mold is an object
- Cake colors can vary even though they come from the same mold (objects have independent properties)

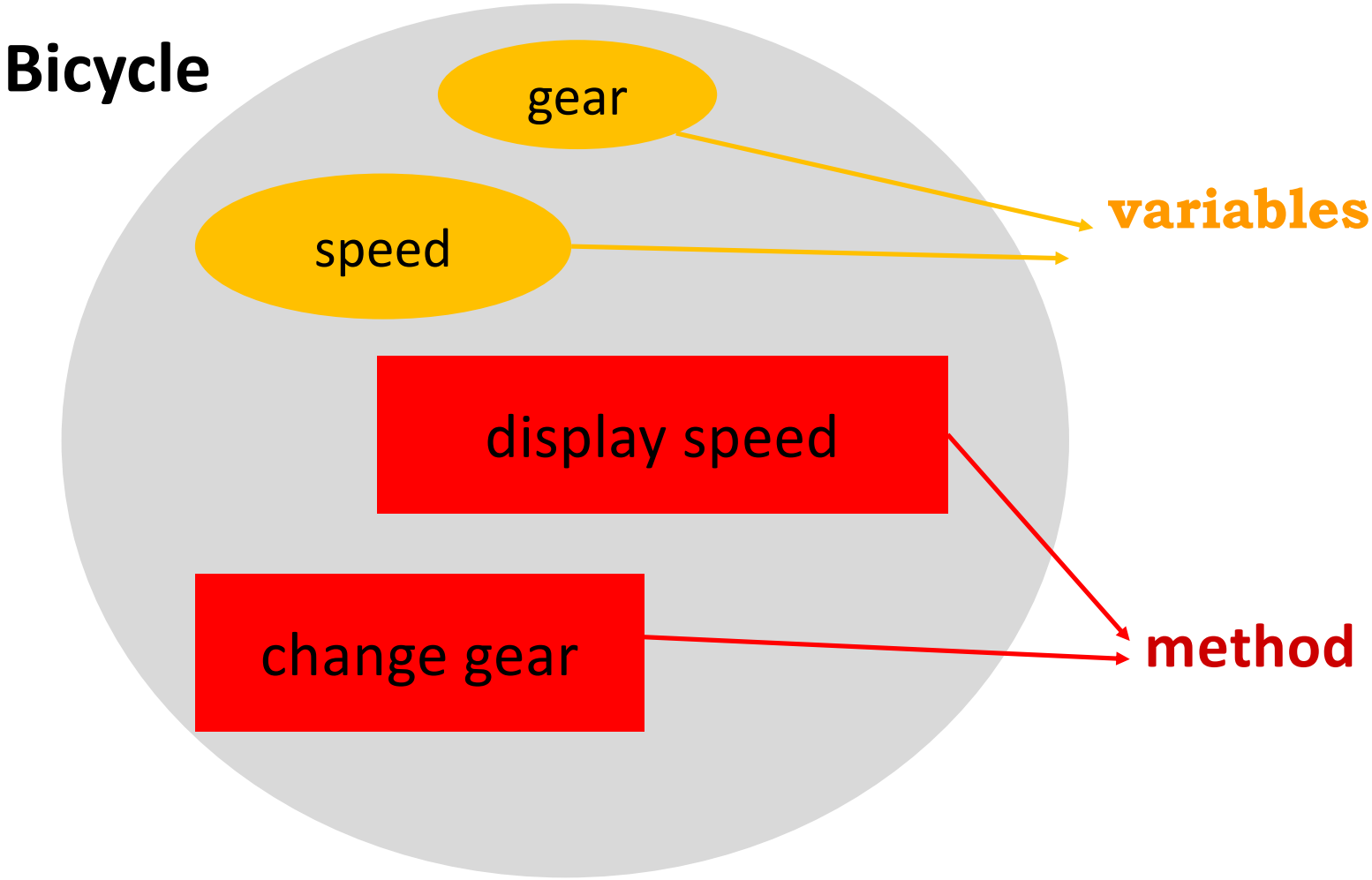


Class with Attributes **Objects with Values**

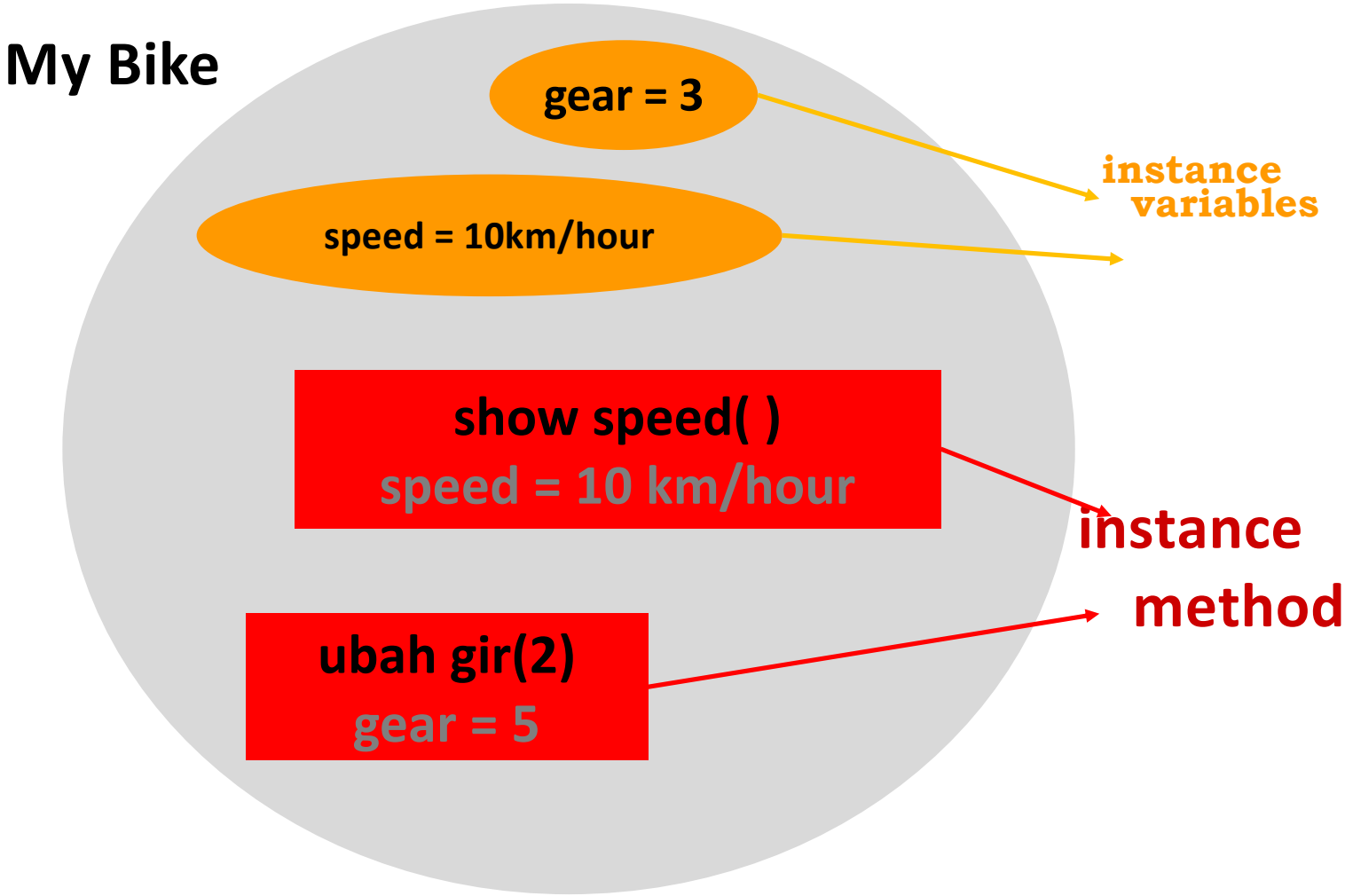
Classes and Objects (continued)



Class = Method + Variable



Object = Method + Variable which has value



Class / Objek Properties

- Atribut

  Data which describes class/object .

  On generally defined with *access modifier* **private** (concept *information hiding*).

  - Data with basic data types : integer , float , boolean , char
  - Compound data : arrays, records , etc.
  - Another class

- Layanan / Metode (Method)

  Procedure or **function** which characteristic or behavior from class / object .

  On generally defined with *access modifier* **public**.

  - Constructor : naming The same with Name his class
  - Accessor : naming started with say get
  - Mutator : naming started with say set
  - Procedure or function specific : naming free
  - Main program : must use Name main()

JAVA INTRODUCTION

What is Java?

- A pure object-oriented programming language, built on the best of C++, Simula, and other OOP languages.
- Created by James Gosling at Sun Microsystems in 1991.
- **Current LTS version: Java 21 (released 2023). Java 17 is the minimum for modern frameworks.**

Java in 2026 — Key Facts

✓ 99% of enterprise organizations actively use Java

✓ 23+ million active Java developers worldwide

✓ #3 in TIOBE Index (Jan 2026)

✓ Powers Fortune 500 companies across finance, healthcare, cloud & mobile

Java Characteristics

Language Design

- Simple
- Object Oriented
- Dynamic

Platform & Portability

- Architecture Neutral
- Portable
- Interpreted

Production-Grade

- Secure
- Distributed
- Multithreaded
- High Performance

Java Programming Tools

- **You need two things to program in Java:**

1

JDK : Java Development Kit

Includes the compiler (javac), runtime (JVM), and standard libraries.

Download: https://www.oracle.com/java/technologies/downloads/ (Java 21 LTS recommended)

2

Code Editor / IDE

Recommended for this course:
- IntelliJ IDEA (Community Edition, free)
- VS Code + Java Extension Pack

Alternatives: Eclipse, NetBeans

Setting Up Java — Step by Step

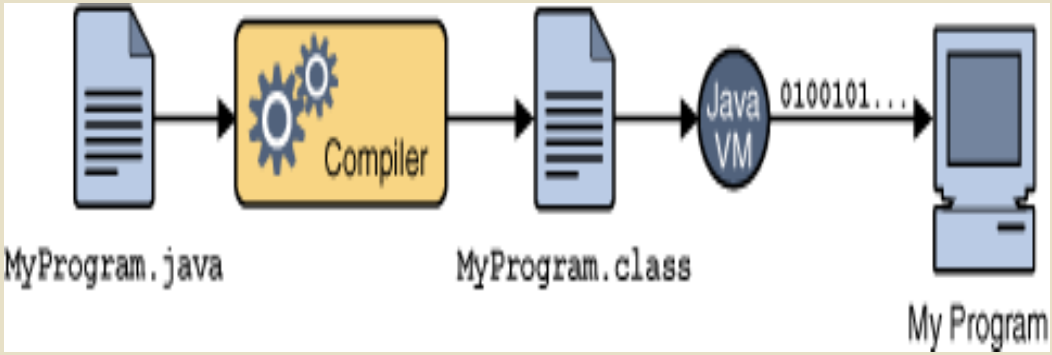
⚠ **Use Java 21 LTS. Old versions will not work with modern tools and frameworks.**

1	**Download JDK 21**	Go to oracle web→ Download Temurin 21 (LTS) Choose your OS: Windows / macOS / Linux
2	**Run the Installer**	Follow the installer prompts. Modern installers set the PATH automatically - check the "Set JAVA_HOME" option.
3	**Verify Installation (Terminal / Command Prompt)**	java -version javac -version Both should show version 21.x.x
4	**Install Your IDE**	IntelliJ IDEA Community (free): https://www.jetbrains.com/idea/ OR VS Code + "Extension Pack for Java"
5	**Create & Run HelloWorld.java**	Your task this week - bring your running output

Java Language (Compiler + Interpreter)

javac
(Java Compiler)

java
(Java Interpreter)



How Java Works?

```
public class Hello
    public static void main(String[] args){
        System.out.println("Hello World!");
    }
}
```

Hello.java

javac (java compiler)

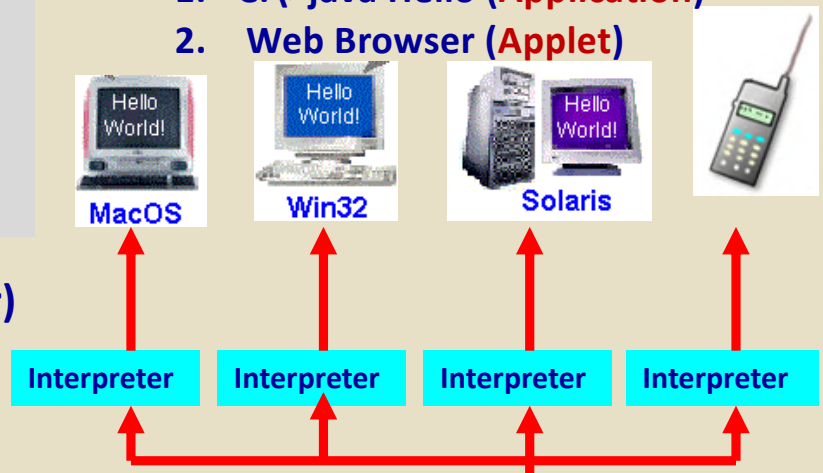


The compiler produces
Bytecode (Class)

Hello.class

Run with:

1. C:\>java Hello (Application)
2. Web Browser (Applet)

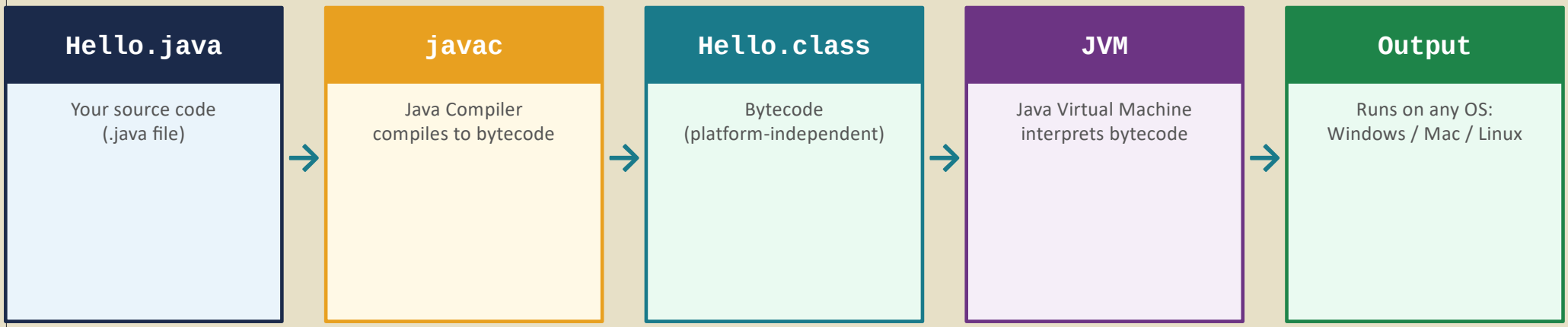




Web Server

Write Once Run Everywhere !

Java: Compiler + Interpreter (Write Once, Run Everywhere)




```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

Terminal Commands

```
# Step 1: Compile
$ javac Hello.java    →  creates Hello.class

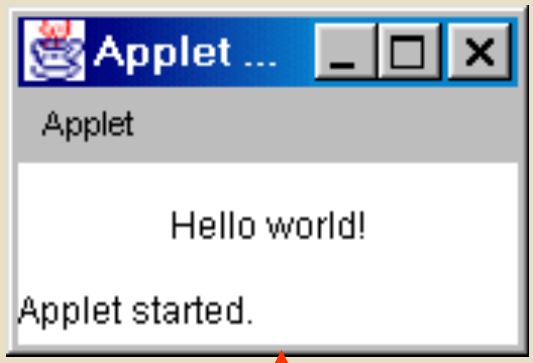
# Step 2: Run
$ java Hello          →  Hello World!
```

⚠ Applets (running Java in browsers) have been removed since Java 17 and no longer work in modern browsers. We focus on standalone console and GUI applications.

Compile and Run Java Applet

```
import java.applet.*;
import java.awt.*;

public class HelloWorld extends Applet {
    public void paint(Graphics g) {
        g.drawString("Hello world!",50,25);
    }
}
```




```
C:\appletviewer Hello.html
```

```
C:\javac HelloWorld.java
```

```
<HTML>
<HEAD>
<TITLE>A Simple Program</TITLE>
</HEAD>
<BODY>
Here is the output of my program:
<APPLET CODE="HelloWorld.class" WIDTH=150
HEIGHT=25>
</APPLET>
</BODY>
</HTML>
```

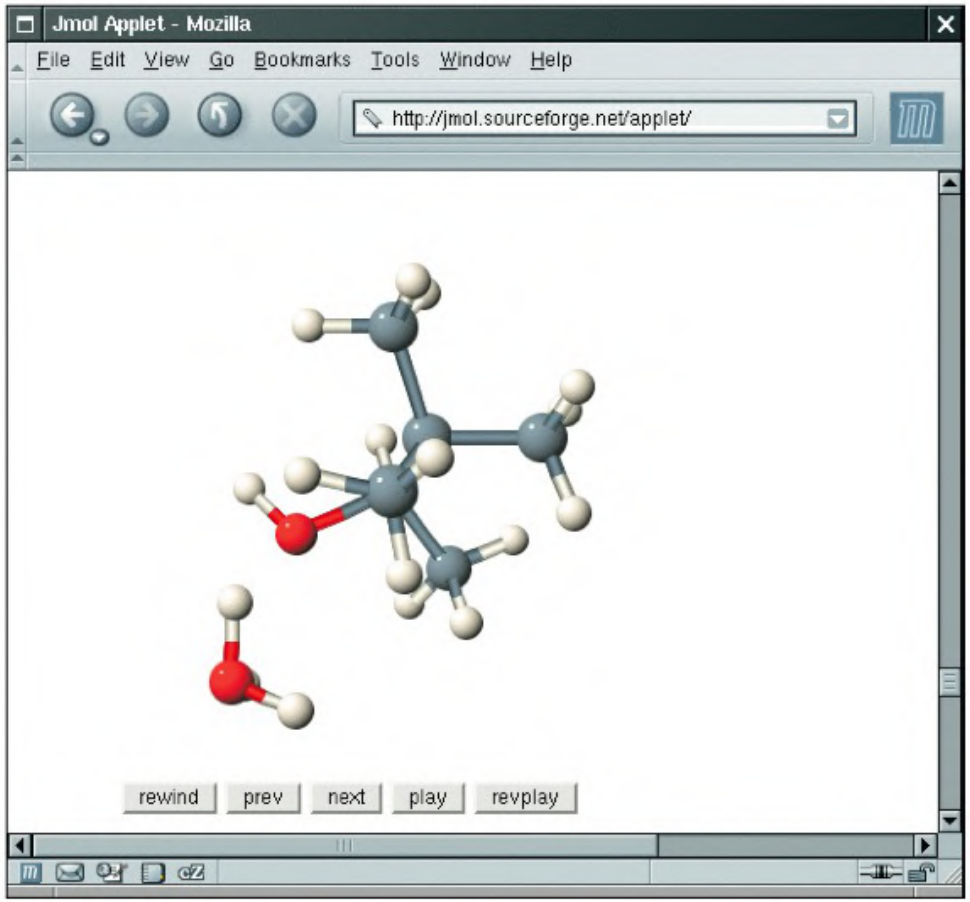


Figure 6 An Applet for Visualizing Molecules Running in a Browser (*http://jmol.sourceforge.net/applet/*)

APPLET ON A WEB PAGE

Java Application

- Stand alone
- Applets
- Windows-based application (GUI)
- Java Servlet
- Java Server Pages (JSP)
- Java Beans and Enterprise Java Beans
- Java Micro Edition → Android Java

Writing Jawa Programmes

- Variation of programes:
  1. Text-Based Console Application → using non-GUI libraries in Java, Text-based input/output through the terminal. What we use for Weeks 1-midtest.
  2. GUI Application → Visual windows, buttons, and forms. Uses JavaFX or Swing libraries. Covered after midtest

- A class can be executed because it has a main method

  public static void main(String[] args)

  → Java programmes start here!

JAVA PROGRAM

- Can be written in a **single** .java file:
  - Contains one class
  - Contains several classes
- Written in **multiple** .java files:
  - One file one class
  - One file multiple classes
- There can only be one main function in each object programme.

Class Declaration

```
[public] class ClassName {
  // Atribute Declaration
  ...

  // method definiton
  <constructor>
  <accessor>
  <mutator>
  <other function>

  // Main Program
  public static void main(String arg[]) {
    ...
  }
}
```

Review : Class/Object Property

- Atribut

  Data which describes class/object .

  On generally defined with *access modifier* **private** (concept *information hiding*).

  - Data with basic data types : integer , float , boolean , char
  - Compound data : arrays, records , etc.
  - Another class

- Method

  Procedure or **function** which characteristic or behavior from class / object .

  On generally defined with *access modifier* **public**.

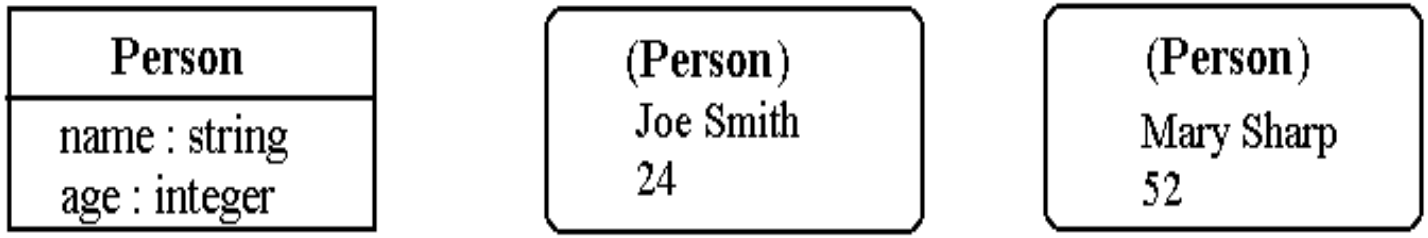
  - Constructor : naming The same with Name his class
  - Accessor : naming started with say get
  - Mutator : naming started with say set
  - Procedure or function specific : naming free
  - Main program : must use Name main()

https://www.w3schools.com/java/java_class_attributes.asp

https://www.w3schools.com/java/java_class_methods.asp

Attributes

- Variables surrounding a class, with data values that can be specified in the object.
- Variables are used to store values that will later be used in the program
- Variables have a type , name and value
- Name, age, and weight are attributes (variables) of the Person



Class with Attributes **Objects with Values**

METHOD : CONSTRUCTOR

- Functions that will be run/executed automatically once an object is created.
- Given the same name as the class name.
- Constructor is a method that is executed to initialize the value of an object attribute with a *default* value.
- Copy Constructor (Cctor) is a method that is executed to initialize the value of an object attribute based on certain parameters.

Example :

```
public Rectangular() {
  // statement
}

public Rectangular(int length, int width) {
  // statement
}
```

```
11  public class Ayah {
12      private String nama;
13
14      public Ayah(){//constructor
15          nama = "default";
16      }
17
18      public Ayah(String nama){//copy constructor
19          this.nama = nama;
20      }
21
22      public String getNama(){...}
25      public void setNama(String input){...}
33  }
```

```
16      public static void main(String[] args) {
17          // TODO code application logic here
18          Ayah a1 = new Ayah();
19          Ayah a2 = new Ayah("Keren");
20
21          System.out.println("Halo " + a1.getNama() + " dan " + a2.getNama());
22      }
```

Method: Constructor

- Runs automatically when an object is created (new ClassName()).
- Always has the exact same name as its class.
- **Two common forms:**
  - Default constructor : sets attributes to default values (no parameters).
  - Parameterized constructor : initializes attributes from given values.

```
// Default constructor
public Rectangular() {
    this.length = 0;
    this.width  = 0;
}

// Parameterized constructor
public Rectangular(int length, int width) {
    this.length = length;
    this.width  = width;
}
```

```
// In main():
Rectangular r1 = new Rectangular();
// r1.length = 0, r1.width = 0

Rectangular r2 = new Rectangular(10, 5);
// r2.length = 10, r2.width = 5
```

METHOD : SELECTOR/ACCESSOR

- A function that has a special use for retrieving values from attributes owned by an object.
- On generally use naming with the format : **getAttributeName** :

Example:

```
public int getLength() {
   return (this.length);
}
```

```
11  public class Ayah {
12      private String nama;
13
14      public Ayah(){//constructor
15          nama = "default";
16      }
17
18      public Ayah(String nama){//copy constructor
19          this.nama = nama;
20      }
21
22      public String getNama(){...}
25      public void setNama(String input){...}
33  }
```

```
16      public static void main(String[] args) {
17          // TODO code application logic here
18          Ayah a1 = new Ayah();
19          Ayah a2 = new Ayah("Keren");
20
21          System.out.println("Halo " + a1.getNama() + " dan " + a2.getNama());
22      }
```

- A function that has a special use for give values to attributes owned by an object
- On generally use naming with the format : **setAttributeName**

Example:

```
public void setLength(int length) {
   this.length = length;
}
```

METHOD : MUTATOR

```
11  public class Ayah {
12      private String nama;
13
14      public Ayah(){//constructor
15          nama = "default";
16      }
17
18      public Ayah(String nama){//copy constructor
19          this.nama = nama;
20      }
21
22      public String getNama(){...}
25      public void setNama(String input){...}
33  }
```

```
16      public static void main(String[] args) {
17          // TODO code application logic here
18          Ayah a1 = new Ayah();
19          Ayah a2 = new Ayah("Keren");
20
21          System.out.println("Halo " + a1.getNama() + " dan " + a2.getNama());
22      }
```

Methods: Accessor (Getter) & Mutator (Setter)

📖 Accessor (Getter)

- Retrieves the value of a private attribute.
- **Naming convention: getAttributeName()**
- Returns the attribute value : has a return type.

```
public int getLength() {
    return this.length;
}

public String getName() {
    return this.name;
}
```

✏️ Mutator (Setter)

- Assigns a new value to a private attribute.
- **Naming convention: setAttributeName()**
- Returns void : it sets, never returns.

```
public void setLength(int length) {
    this.length = length;
}

public void setName(String name) {
    this.name = name;
}
```

Keyword: this

this refers to the current object, the specific instance a method is operating on.

Most common use: disambiguate when a parameter name matches an attribute name.

✅ **With this (recommended : clear and unambiguous)**

```
void fillData(String colour,
              int productionYear) {
    this.colour = colour;
    this.productionYear = productionYear;
    // this.colour  →  the object's field
    // colour       →  the parameter
}
```

⚠️ **Without this (works only if names differ)**

```
void fillData(String c,
              int yp) {
    colour = c;
    productionYear = yp;
    // No ambiguity here, but
    // using different names is
    // harder to read.
}
```

💡 Rule of thumb: always use this.attributeName inside constructors and setters, it makes your intent unambiguous and your code self-documenting.

FUNCTION main()

- Works as the **main program** that will be become **starting point for execution** .
- Writing format :

  public static void main (String arg []) { ... }

  - **public** : function can accessible from in nor outside class
  - **static** : the same for all instant from class
  - **void** : function that no give *return value*
  - **arg []** : array of Strings to be argument function (taken fr
- Contents main() function on generally :
  - Creation (instantiation) of objects
  - Object manipulation : copying , use

```
11  public class Ayah {
12      private String nama;
13
14      public Ayah(){//constructor
15          nama = "default";
16      }
17
18      public Ayah(String nama){//copy constructor
19          this.nama = nama;
20      }
21
22      public String getNama(){...}
25      public void setNama(String input){...}
33  }
```

```
16      public static void main(String[] args) {
17          // TODO code application logic here
18          Ayah a1 = new Ayah();
19          Ayah a2 = new Ayah("Keren");
20
21          System.out.println("Halo " + a1.getNama() + " dan " + a2.getNama());
22      }
```

Java Class Structure

```
public class ClassName {

    // 1. Attribute Declarations (usually private)
    private String name;
    private int age;

    // 2. Constructor
    public ClassName() { ... }
    public ClassName(String name, int age) { ... }

    // 3. Accessor (getter)
    public String getName() { return this.name; }

    // 4. Mutator (setter)
    public void setName(String name) { this.name = name; }

    // 5. Other methods
    public void display() { ... }

    // 6. Main program (entry point)
    public static void main(String[] args) {
        ClassName obj = new ClassName("Alice", 20);
        System.out.println(obj.getName());
    }
}
```

Attributes

Declared private — data hidden from outside (encapsulation)

Constructor

Same name as class.
Initializes object attributes.

Accessor

Named getXxx()
Returns attribute value

Mutator

Named setXxx()
Sets attribute value

main()

Entry point of the program.
Creates objects, uses them.

Object Creation

- Reference or variables declaration :

  Student m;

- Allocation object to variables :

  m = new Student ();

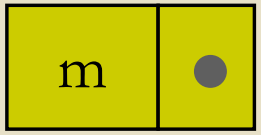
  Or

- Declaration and allocation object :

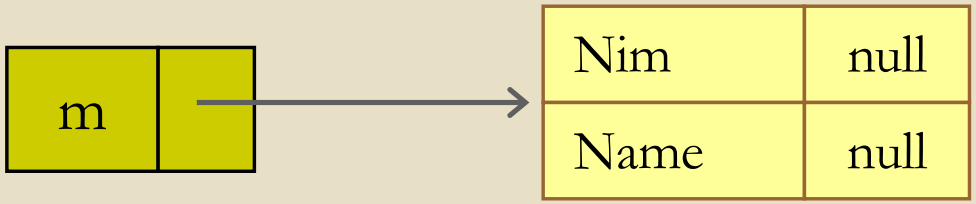
  Student m = new Student ();

Object Creation (continued)

- Student m ; // declaration variable

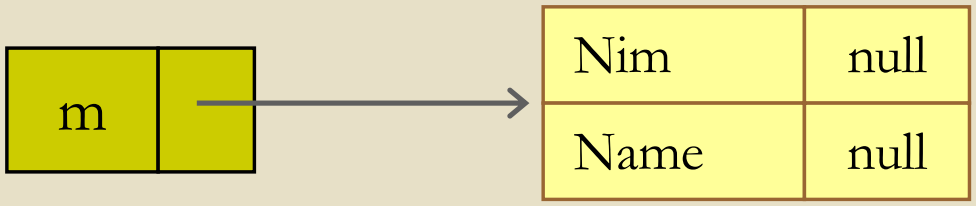


- m = new Student (); // allocation object

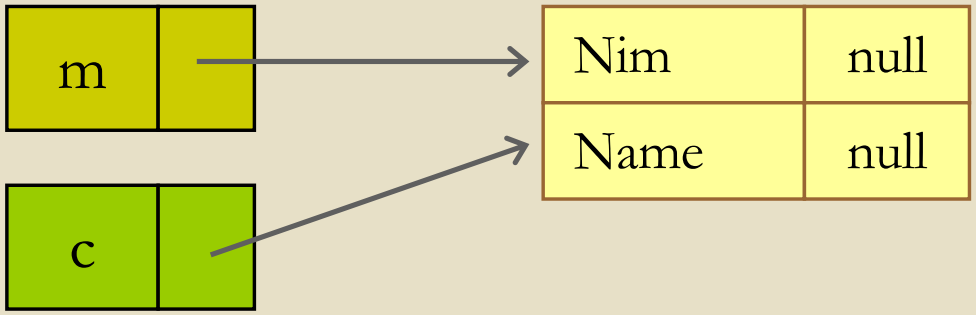


Copying (Copy) Objects

- Student m = new Student ();



- c = m;



OBJECT USE

- Done with *message passing* to object .
- Call a method owned by an object.

Example :

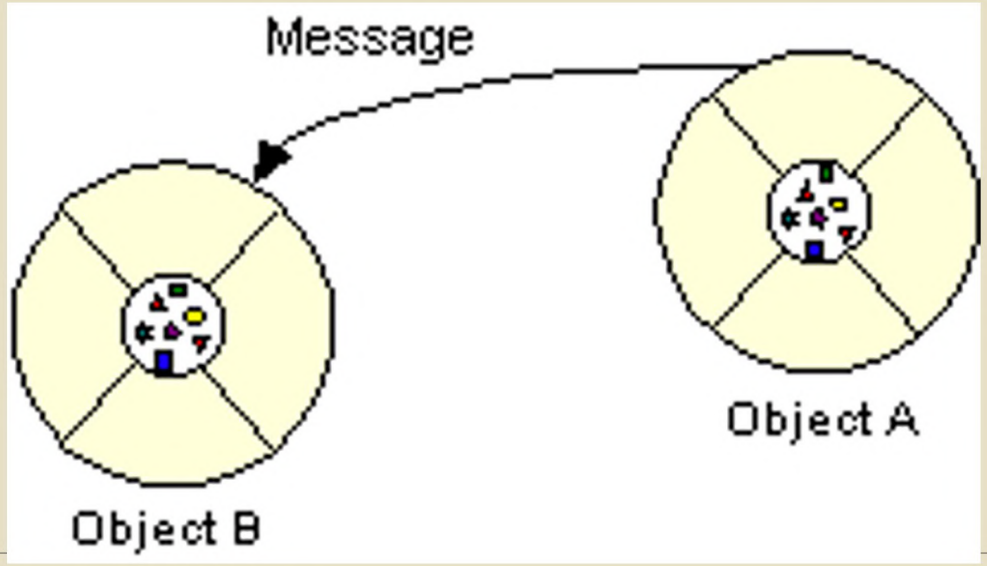
```
s.setLength (17);
s.setWidth (8);
System.Out.Println (“ Area = “ + s.Area () );
```

Name object

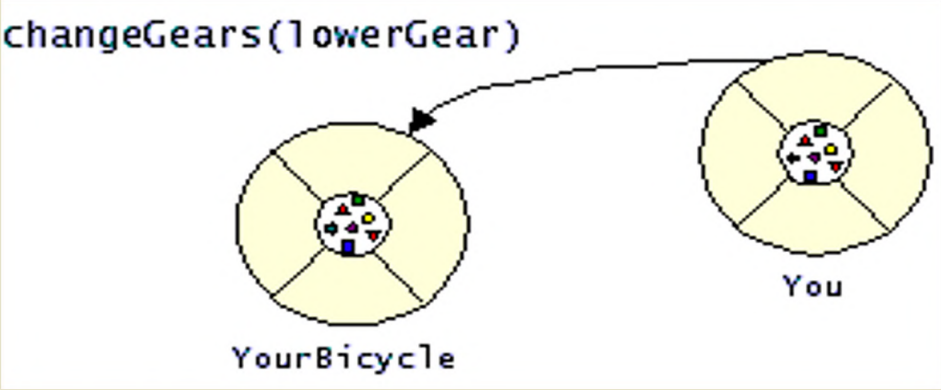
method name

Concept: Parameters

- A bicycle is only useful if there are other objects that interact with it.
- Software objects interact and communicate with other objects by sending messages.
- A message is a method, and the information in the message is known as parameters.



Message Delivery and Parameters



1. **You** → object sender
2. **YourBicycle** → receiver object
3. **changeGears** → message in the form of an executed method
4. **lowerGear** →parameter required method (message) to be executed

Creating Classes, Objects and Calling Attributes

Car.java

```
public class Car {
Color strings;
intProductionYear;
}
```

Cars in Action.java

```
public class ActionCar {
public static void main(String[] args){
 // Create objects
        My car car = new Car();

       /* call attribute and assign value */
        mycar.color = "Black";
        mycar.yearProduction = 2006;
      System.out.println("Color: " + mycar.color );
      System.out.println("Year: " + mycar.yearProduction) ;
}
}
```

```
class QuadrilateralDB {

// Declaration attribute
private float length , width ;

// Declaration service (method)
// Function For count wide facet four
public float Width() { return ( length * width ); }

// Function For count around facet four
public float Perimeter () { return (2*( length+width )); }

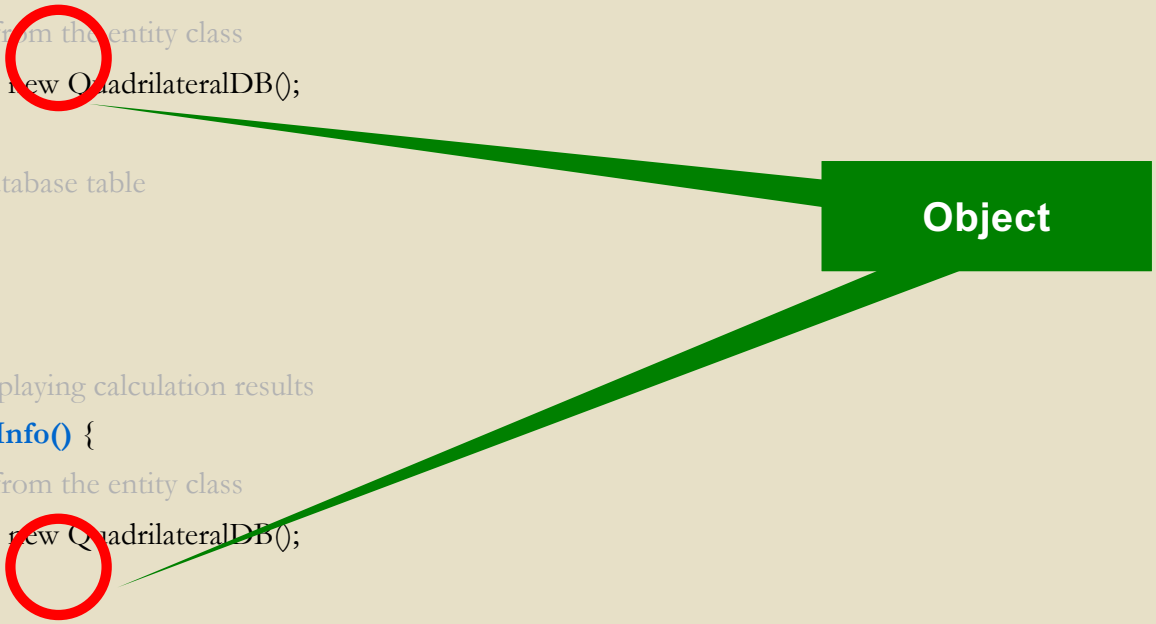
// Procedure For save data to database table
public void AddData (float p, float l) {
…
}

// Procedure For retrieve data from database table
public void GetData () {
…
}
}
```

```
class QuadrilateralCtrl {
// Service declaration (method)
// Procedure for recording data to a database table
public void RecordData(float pjg, float lbr) {
// Create an object from the entity class
QuadrilateralDB S = new QuadrilateralDB();

// Record data to database table
S.AddData(pjg, lbr);
}

// Procedure for displaying calculation results
public void DisplayInfo() {
// Create an object from the entity class
QuadrilateralDB S = new QuadrilateralDB();
...
}
}
```



Putting It Together : A Complete Example

```java
// File: Car.java
public class Car {
    private String colour;
    private int    productionYear;

    public Car() {
        this.colour         = "Unknown";
        this.productionYear = 0;
    }
    public Car(String colour, int productionYear) {
        this.colour         = colour;
        this.productionYear = productionYear;
    }
    public String getColour()         { return this.colour;
}
    public int    getProductionYear() { return
this.productionYear; }
    public void   setColour(String c) { this.colour = c; }
    public void   setProductionYear(int y) {
this.productionYear = y; }
}
```

```java
// File: CarDemo.java
public class CarDemo {
    public static void main(String[] args) {

        // Create objects
        Car c1 = new Car("Black", 2023);
        Car c2 = new Car();

        // Use accessors
        System.out.println(c1.getColour());
        // Output: Black

        // Use mutators
        c2.setColour("Red");
        c2.setProductionYear(2026);
        System.out.println(c2.getColour());
        // Output: Red
    }
}
```

Practice Exercises

BangunDatar (2D Shapes) & BangunRuang (3D Shapes)

Exercise 1 : Class BangunDatar (2D Shapes)

Class: BangunDatar

- **Attributes: side (for square), length, width (for rectangle)**
- **Constructors:**
  - Default constructor (no parameters)
  - Constructor with side parameter (square)
  - Constructor with length & width parameters (rectangle)
- **Accessor & Mutator for: side, length, width**
- **Methods:**
  - calculateSquareArea()
  - calculateRectangleArea()
  - calculateSquarePerimeter()
  - calculateRectanglePerimeter()

Class: BangunDatarDemo

- **Contains main() method**
- **Create objects:**
  - square → using side constructor
  - rectangle1 → using default constructor, then setters
  - rectangle2 → using length & width constructor
- **Calculate and print area & perimeter of each object**
- **Output must be descriptive, e.g.:**
  - "Area of rectangle with length 10 and width 5 = 50"
  - "Perimeter of square with side 7 = 28"

Exercise 2 : Class BangunRuang (3D Shapes)

Class: BangunRuang

- **Attributes: side, length, width, height, radius**
- **Constructors:**
  - Default constructor
  - Constructor with side (cube)
  - Constructor with radius (sphere)
  - Constructor with length, width & height (box)
- **Accessor & Mutator for all 5 attributes**
- **Methods:**
  - calculateCubeSurfaceArea()
  - calculateBoxSurfaceArea()
  - calculateSphereSurfaceArea()
  - calculateCubeVolume()
  - calculateBoxVolume()
  - calculateSphereVolume()

Class: BangunRuangDemo

- **Contains main() method**
- **Create objects:**
  - cube → using side constructor
  - box1 → using default constructor, then setters
  - box2 → using length, width & height constructor
  - sphere → using radius constructor
- **Calculate and print volume & surface area of each:**
  - "Volume of box (L=10, W=5, H=7) = 350.0"
  - "Surface area of cube (side=4) = 96.0"
  - "Volume of sphere (radius=3) = 113.1"

Thank You

AI Policy: You may use AI tools to help understand some theory or concepts.
Rule: you must be able to explain every line of code you submit